

# **SEMINARARBEIT**

Im Studiengang FH Technikum Wien, Bachelorstudiengang BIF  
Lehrveranstaltung BIF-VZ-5-WS2025-INFC-EN

## **Terraform Workshop Protocol**

Ausgeführt von: David Veigel

Kevin Forter

Personenkennzeichen: 2310257030,  
2500257003

BegutachterIn:

Wien, den 18. Dezember 2025

# Inhaltsverzeichnis

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Workshop 1: Terraform Basics</b>                     | <b>1</b>  |
| 1.1      | Configuration . . . . .                                 | 1         |
| 1.1.1    | Provider and Region . . . . .                           | 1         |
| 1.1.2    | VPC and Subnets . . . . .                               | 1         |
| 1.1.3    | Security Group . . . . .                                | 1         |
| 1.1.4    | EC2 Instance . . . . .                                  | 2         |
| 1.1.5    | Output and Verification . . . . .                       | 3         |
| 1.2      | Execution . . . . .                                     | 4         |
| <b>2</b> | <b>Workshop 2: Advanced Networking</b>                  | <b>5</b>  |
| 2.1      | Network Infrastructure . . . . .                        | 5         |
| 2.1.1    | VPC and Internet Gateway . . . . .                      | 5         |
| 2.1.2    | Subnet and Routing . . . . .                            | 7         |
| 2.2      | Compute and Access . . . . .                            | 8         |
| 2.2.1    | Security Group . . . . .                                | 8         |
| 2.2.2    | Network Interface and Elastic IP . . . . .              | 9         |
| 2.2.3    | EC2 Instance . . . . .                                  | 9         |
| 2.3      | Execution . . . . .                                     | 10        |
| <b>3</b> | <b>Workshop 3: Load Balancing and High Availability</b> | <b>11</b> |
| 3.1      | Networking Setup . . . . .                              | 11        |
| 3.1.1    | VPC and Internet Gateway . . . . .                      | 11        |
| 3.1.2    | Subnets in Multiple AZs . . . . .                       | 12        |
| 3.1.3    | Routing . . . . .                                       | 14        |
| 3.2      | Security Groups . . . . .                               | 15        |
| 3.2.1    | Load Balancer Security Group . . . . .                  | 15        |
| 3.2.2    | EC2 Security Group . . . . .                            | 15        |
| 3.3      | Load Balancer Configuration . . . . .                   | 16        |
| 3.3.1    | Application Load Balancer . . . . .                     | 16        |
| 3.3.2    | Target Group and Listener . . . . .                     | 17        |
| 3.4      | Compute Resources . . . . .                             | 18        |
| 3.4.1    | EC2 Instances . . . . .                                 | 18        |
| 3.4.2    | Target Group Attachment . . . . .                       | 20        |
| 3.5      | Verification . . . . .                                  | 20        |

|       |                                         |    |
|-------|-----------------------------------------|----|
| 3.6   | Automation and CI/CD . . . . .          | 21 |
| 3.6.1 | GitHub Configuration . . . . .          | 21 |
| 3.6.2 | Terraform Cloud Configuration . . . . . | 22 |
| 3.6.3 | CI/CD Pipeline . . . . .                | 22 |
| 3.7   | Source Code . . . . .                   | 29 |

# 1 Workshop 1: Terraform Basics

In this workshop, we set up a simple web server on AWS using Terraform. The goal is to provision an EC2 instance in the default VPC, install an Apache web server using user data, and verify that it is reachable.

## 1.1 Configuration

### 1.1.1 Provider and Region

We configure the AWS provider to use the `us-east-1` region, as specified in the requirements.

```
provider "aws" {  
    region = "us-east-1"  
}
```

### 1.1.2 VPC and Subnets

Instead of creating a new network, we use the existing default VPC and its subnets. This simplifies the setup for this initial workshop.

```
data "aws_vpc" "default" {  
    default = true  
}  
  
data "aws_subnets" "default_vpc_subnets" {  
    filter {  
        name     = "vpc-id"  
        values = [data.aws_vpc.default.id]  
    }  
}
```

### 1.1.3 Security Group

To make our web server accessible, we create a security group. We allow inbound TCP traffic on port 80 (HTTP) from any IP address (`0.0.0.0/0`). We also allow all outbound traffic to ensure the server can download updates and packages.

```

resource "aws_security_group" "web_sg" {
  name          = "tf-web-sg"
  description   = "Allow_HTTP_in,_all_egress"
  vpc_id        = data.aws_vpc.default.id

  ingress {
    description      = "HTTP"
    from_port        = 80
    to_port          = 80
    protocol          = "tcp"
    cidr_blocks       = ["0.0.0.0/0"]
  }

  egress {
    description      = "All_egress"
    from_port        = 0
    to_port          = 0
    protocol          = "-1"
    cidr_blocks       = ["0.0.0.0/0"]
  }
}

```

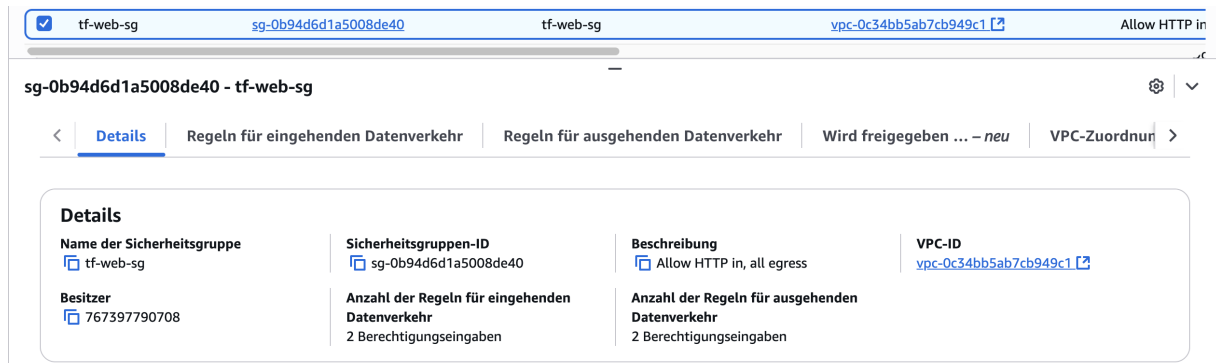


Abbildung 1: Security Group Configuration in AWS Console

### 1.1.4 EC2 Instance

We provision an EC2 instance using the `t2.micro` instance type, which is eligible for the free tier. We dynamically fetch the latest Ubuntu 22.04 LTS AMI. The `user_data` script is used to automatically install and start the Apache web server upon launch.

```

resource "aws_instance" "web" {
  ami          = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  subnet_id    = data.aws_subnets.default_vpc_subnets.ids[0]
}

```

```

vpc_security_group_ids      = [aws_security_group.web_sg.id]
associate_public_ip_address = true

user_data = <<-EOF
#!/bin/bash
sudo apt-get update
sudo apt-get install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
echo "<h1>Hello_World</h1>" | sudo tee /var/www/html/index.html
EOF

tags = {
  Name = "tf-apache"
}

```

**Instances (1/6)** Info

Verbinden Instance-Status Aktionen Instances starten

Instance nach Attribut oder Tag (case-sensitive) suchen

| Name                                          | Instance-ID         | Instance-Status | Instance-Typ | Statusüberprüfun | Alarmstatus     | Availability Zone | Öffentlich |
|-----------------------------------------------|---------------------|-----------------|--------------|------------------|-----------------|-------------------|------------|
| <input checked="" type="checkbox"/> tf-apache | i-0d752bc52c5dc8ada | Lauft           | t2.micro     | 2/2-Prüfungen be | Alarme anzeigen | us-east-1c        | ec2-54-227 |
| <input type="checkbox"/> Swarm-Node-01        | i-0a103f7322becf0f1 | Lauft           | t3.small     | 3/3 Prüfungen be | Alarme anzeigen | us-east-1d        | -          |
| <input type="checkbox"/> Swarm-Node-02        | i-06cdf270d7aeb99f  | Lauft           | t3.micro     | 3/3 Prüfungen be | Alarme anzeigen | us-east-1d        | -          |
| <input type="checkbox"/> Swarm-Node-03        | i-08a6cf84af35b8980 | Lauft           | t3.micro     | 3/3 Prüfungen be | Alarme anzeigen | us-east-1d        | -          |

**i-0d752bc52c5dc8ada (tf-apache)**

Details Status und Alarme Überwachung Sicherheit Netzwerk Speicher Tags

**▼ Instance-Zusammenfassung** Info

|                                                                                     |                                                                        |                                                                                                                                            |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Instance-ID</b><br>i-0d752bc52c5dc8ada                                           | <b>Öffentliche IPv4-Adresse</b><br>54.227.33.245   offene Adresse      | <b>Private IPv4-Adressen</b><br>172.31.18.248                                                                                              |
| <b>IPv6-Adresse</b><br>-                                                            | <b>Instance-Status</b><br>Lauft                                        | <b>Öffentlicher DNS</b><br>ec2-54-227-33-245.compute-1.amazonaws.com   offene Adresse                                                      |
| <b>Hostname-Typ</b><br>IP-Name: ip-172-31-18-248.ec2.internal                       | <b>Private IP DNS-Name (nur IPv4)</b><br>ip-172-31-18-248.ec2.internal | <b>Elastic IP-Adressen</b><br>-                                                                                                            |
| <b>DNS-Name der Antwort für private Ressourcen</b><br>-                             | <b>Instance-Typ</b><br>t2.micro                                        | <b>AWS Compute Optimizer-Treffer</b><br>Melden Sie sich für AWS Compute Optimizer an, um Empfehlungen zu erhalten.   Weitere Informationen |
| <b>Automatisch zugewiesene IP-Adresse</b><br>54.227.33.245 [Öffentliche IP-Adresse] | <b>VPC-ID</b><br>vpc-0c34bb5ab7cb949c1                                 |                                                                                                                                            |

Abbildung 2: EC2 Instance Running in AWS Console

## 1.1.5 Output and Verification

Finally, we output the public DNS of the instance to easily access it. We also include a `null_resource` to verify that the web server is actually reachable via HTTP.

```

output "instance_public_dns" {

```

```
description = "Public_DNS_of_the_EC2_instance"
value       = aws_instance.web.public_dns
}
```

## 1.2 Execution

We initialize the Terraform project to download the necessary providers.

```
> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching ">= 5.0.0"...
- Finding latest version of hashicorp/null...
- Installing hashicorp/aws v6.20.0...
- Installed hashicorp/aws v6.20.0 (signed by HashiCorp)
- Installing hashicorp/null v3.2.4...
- Installed hashicorp/null v3.2.4 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Abbildung 3: Terraform Init

Then we apply the configuration to create the resources.

```

> terraform apply -auto-approve
data.aws_vpc.default: Reading...
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 4s [id=ami-0c398cb65a93047f2]
data.aws_vpc.default: Read complete after 7s [id=vpc-0c34bb5ab7cb949c1]
data.aws_subnets.default_vpc_subnets: Reading...
data.aws_subnets.default_vpc_subnets: Read complete after 1s [id=us-east-1]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                        = "ami-0c398cb65a93047f2"
  + arn                       = (known after apply)
  + associate_public_ip_address = true
  + availability_zone          = (known after apply)
  + disable_api_stop           = (known after apply)
  + disable_api_termination    = (known after apply)
  + ebs_optimized              = (known after apply)
  + enable_primary_ipv6        = (known after apply)
  + force_destroy              = false
  + get_password_data          = false
  + host_id                    = (known after apply)
  + host_resource_group_arn    = (known after apply)
  + iam_instance_profile       = (known after apply)
  + id                         = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle         = (known after apply)
  + instance_state             = (known after apply)
  + instance_type              = "t2.micro"
  + ipv6_address_count          = (known after apply)
  + ipv6_addresses             = (known after apply)
  + key_name                    = (known after apply)
  + monitoring                  = (known after apply)
  + outpost_arn                = (known after apply)
  + password_data              = (known after apply)
  + placement_group            = (known after apply)
  + placement_group_id         = (known after apply)
  + placement_partition_number = (known after apply)
  + primary_network_interface_id = (known after apply)
  + private_dns                 = (known after apply)
  + private_ip                 = (known after apply)
  + public_dns                 = (known after apply)
  + public_ip                  = (known after apply)
  + region                     = "us-east-1"
  + secondary_private_ips       = (known after apply)
  + security_groups             = (known after apply)
  + source_dest_check           = true

```

Abbildung 4: Terraform Apply

## 2 Workshop 2: Advanced Networking

In the second workshop, we move away from the default VPC and build a custom network infrastructure. This includes creating a VPC, subnets, routing tables, and an Internet Gateway. We also assign a persistent Elastic IP to our web server.

### 2.1 Network Infrastructure

#### 2.1.1 VPC and Internet Gateway

We create a new VPC with the CIDR block `10.0.0.0/16`. To enable communication with the internet, we attach an Internet Gateway to this VPC.



```
resource "aws_vpc" "web_vpc" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = { Name = "tf-web-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.web_vpc.id
  tags = { Name = "tf-web-igw" }
}
```

**vpc-01642c1407e32f458 / tf-web-vpc** Aktionen ▾

| Details <small>Info</small>                                                                                                                                                                                                           |                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>VPC-ID</b><br>vpc-01642c1407e32f458                                                                                                                                                                                                | <b>Status</b><br>Available                                  |
| <b>DNS-Auflösung</b><br>Aktiviert                                                                                                                                                                                                     | <b>Tenancy</b><br>default                                   |
| <b>Haupt-Netzwerk-ACL</b><br>acl-0f155e51d51550947                                                                                                                                                                                    | <b>Standard-VPC</b><br>Nein                                 |
| <b>IPv6 CIDR (Network Border Group)</b><br>-                                                                                                                                                                                          | <b>Nutzungsmetriken für Netzwerkadressen</b><br>Deaktiviert |
| <b>Verschlüsselungssteuerungs-ID</b><br>-                                                                                                                                                                                             | <b>Verschlüsselungssteuerungsmodus</b><br>-                 |
| <b>Öffentlichen Zugriff blockieren</b><br>Aus<br><b>DHCP-Optionssatz</b><br>dopt-0804897584b01e432<br><b>IPv4-CIDR</b><br>10.0.0.0/16<br><b>Regelgruppen für Route-53-Resolver-DNS-Firewall</b><br>Fehler beim Laden der Regelgruppen |                                                             |
| <b>DNS-Hostnamen</b><br>Aktiviert<br><b>Haupt-Routing-Tabelle</b><br>rtb-07e4fa61f19867c7f<br><b>IPv6-Pool</b><br>-<br><b>Besitzer-ID</b><br>767397790708                                                                             |                                                             |

**Ressourcenzuordnung** Info CIDRs Flow-Protokolle Tags Integrationen

**Ressourcenzuordnung** Info Alle Details anzeigen

**VPC**  
Ihr virtuelles AWS-Netzwerk  
tf-web-vpc

**Subnetze (1)**  
Subnetze innerhalb dieser VPC  
us-east-1a  
tf-web-subnet

**Routing-Tabellen (2)**  
Netzwerkdatenverkehr an Ressourcen weiterleiten  
tf-web-rt  
rtb-07e4fa61f19867c7f

Abbildung 5: Custom VPC Created

**igw-0c40f497ee5149ed6 / tf-web-igw** Aktionen ▾

| Details <small>Info</small>                         |                                   |
|-----------------------------------------------------|-----------------------------------|
| <b>Internet-Gateway-ID</b><br>igw-0c40f497ee5149ed6 | <b>Status</b><br>Attached         |
| <b>VPC-ID</b><br>vpc-01642c1407e32f458   tf-web-vpc | <b>Eigentümer</b><br>767397790708 |

**Tags (1)** Tags verwalten

Tags suchen

| Schlüssel | Wert       |
|-----------|------------|
| Name      | tf-web-igw |

Abbildung 6: Internet Gateway Attached

## 2.1.2 Subnet and Routing

We create a subnet (10.0.1.0/24) within our VPC. To allow traffic from the subnet to reach the internet, we create a custom route table that routes all traffic (0.0.0.0/0) to the Internet Gateway, and associate it with our subnet.

```
resource "aws_route_table" "web_rt" {
  vpc_id = aws_vpc.web_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "tf-web-rt" }
}

resource "aws_subnet" "web_subnet" {
  vpc_id            = aws_vpc.web_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "us-east-1a"
  map_public_ip_on_launch = false
  tags = { Name = "tf-web-subnet" }
}

resource "aws_route_table_association" "web_rta" {
  subnet_id      = aws_subnet.web_subnet.id
  route_table_id = aws_route_table.web_rt.id
}
```

**subnet-003febe3900aa2e0f / tf-web-subnet** Aktionen

**Details**

**Subnetz-ID**  
subnet-003febe3900aa2e0f

**IPv4-CIDR**  
10.0.1.0/24

**Verfügbare Zone**  
us-east-1 (us-east-1a)

**Netzwerk-ACL**  
acl-0f155e51d51550947

**Kundeneigene IPv4-Adresse automatisch zuweisen**  
Nein

**IPv6 CIDR-Reservierungen**  
-

**Ressourcenname-DNS-A-AAAA-Datensatz**  
Deaktiviert

**Subnetz-ARN**  
arn:aws:ec2:us-east-1:767397790708:subnet/subnet-003febe3900aa2e0f

**Verfügbare IPv4-Adressen**  
250

**Network Border Group**  
us-east-1

**Standard-Subnetz**  
Nein

**Kundeneigener IPv4-Pool**  
-

**Nur IPv6**  
Nein

**DNS64**  
Deaktiviert

**Status**  
Available

**IPv6-CIDR**  
-

**VPC**  
vpc-01642c1407e32f458 | tf-web-vpc

**Öffentliche IPv4-Adresse automatisch zuweisen**  
Nein

**Stützpunkt-ID**  
-

**Hostname-Typ**  
IP-Name

**Eigentümer**  
767397790708

**Öffentlichen Zugriff blockieren**  
Aus

**IPv6-CIDR-Zuordnungs-ID**  
-

**Routing-Tabelle**  
rtb-09d58ee8e0b88609c | tf-web-rt

**IPv6-Adresse automatisch zuweisen**  
Nein

**IPv4 CIDR-Reservierungen**  
-

**Ressourcenname-DNS-A-Datensatz**  
Deaktiviert

Flow-Protokolle | **Routing-Tabelle** | Netzwerk-ACL | CIDR-Reservierungen | Freigeben | Tags

**Routing-Tabelle:** rtb-09d58ee8e0b88609c / tf-web-rt Routing-Tabellenzuordnung bearbeiten

**Routen (2)**

Suchen Routen filtern

| Bestimmungsort | Ziel                  |
|----------------|-----------------------|
| 10.0.0.0/16    | local                 |
| 0.0.0.0/0      | igw-0c40f497ee5149ed6 |

Abbildung 7: Subnet and Route Table Association

## 2.2 Compute and Access

### 2.2.1 Security Group

Similar to Workshop 1, we create a security group to allow HTTP traffic.

**sg-06bad6ac684893f17 - tf-web-sg** Aktionen

**Details**

**Name der Sicherheitsgruppe**  
tf-web-sg

**Besitzer**  
767397790708

**Sicherheitsgruppen-ID**  
sg-06bad6ac684893f17

**Anzahl der Regeln für eingehenden Datenverkehr**  
2 Berechtigungseingaben

**Beschreibung**  
Allow HTTP in, all egress

**Anzahl der Regeln für ausgehenden Datenverkehr**  
2 Berechtigungseingaben

**VPC-ID**  
vpc-01642c1407e32f458

Regeln für eingehenden Datenverkehr | Regeln für ausgehenden Datenverkehr | Wird freigegeben ... - neu | VPC-Zuordnungen - neu | Tags

**Regeln für eingehenden Datenverkehr (2)** Tags verwalten Regeln für eingehenden Datenverkehr bearbeiten

Suchen

|                          | Name | ID der Sicherheitsgr... | IP-Version | Typ  | Protokoll | Portbereich |
|--------------------------|------|-------------------------|------------|------|-----------|-------------|
| <input type="checkbox"/> | -    | sgr-082dfdda289601aa7   | IPv4       | HTTP | TCP       | 80          |
| <input type="checkbox"/> | -    | sgr-0ceac4166dfa6c64f   | IPv6       | HTTP | TCP       | 80          |

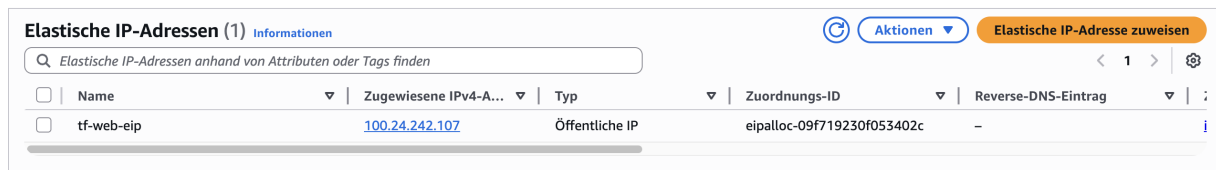
Abbildung 8: Security Group Inbound Rules

## 2.2.2 Network Interface and Elastic IP

Instead of relying on auto-assigned public IPs, we create a Network Interface (ENI) with a fixed private IP (10.0.1.50) and associate an Elastic IP (EIP) with it. This gives our server a static public IP address.

```
resource "aws_network_interface" "web_eni" {
  subnet_id      = aws_subnet.web_subnet.id
  private_ips    = ["10.0.1.50"]
  security_groups = [aws_security_group.web_sg.id]
  tags = { Name = "tf-web-eni" }
}

resource "aws_eip" "web_eip" {
  domain                = "vpc"
  network_interface     = aws_network_interface.web_eni.id
  associate_with_private_ip = "10.0.1.50"
  depends_on            = [aws_internet_gateway.igw]
  tags = { Name = "tf-web-eip" }
}
```



| Name       | Zugewiesene IPv4-Adresse | Typ            | Zuordnungs-ID              | Reverse-DNS-Eintrag |
|------------|--------------------------|----------------|----------------------------|---------------------|
| tf-web-eip | 100.24.242.107           | Öffentliche IP | eipalloc-09f719230f053402c | -                   |

Abbildung 9: Elastic IP Allocation

## 2.2.3 EC2 Instance

We launch the EC2 instance and attach the previously created network interface. This ensures the instance uses our specific network configuration.

```
resource "aws_instance" "web" {
  ami          = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"

  network_interface {
    network_interface_id = aws_network_interface.web_eni.id
    device_index         = 0
  }

  user_data = <<-EOF
    #!/bin/bash
    sudo apt-get update
  >>>EOF
}
```

```

sudo apt-get install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
echo "<h1>Hello_World</h1>" | sudo tee /var/www/html/index.html
EOF

tags = { Name = "tf-apache-instance" }
}

```

The screenshot displays the AWS Management Console's 'Instance-Zusammenfassung' (Instance Summary) page for an EC2 instance with ID 'i-0b44d8e0decc8c1f3' (tf-apache-instance). The page is organized into three columns of key-value pairs, with a top navigation bar containing buttons for 'Verbinden', 'Instance-Status', and 'Aktionen'.

| Instance-Zusammenfassung für i-0b44d8e0decc8c1f3 (tf-apache-instance)                                                        | Öffentliche IPv4-Adresse                                                               | Private IPv4-Adressen                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Instance-ID</b><br>i-0b44d8e0decc8c1f3                                                                                    | 100.24.242.107   <a href="#">offene Adresse</a>                                        | 10.0.1.50                                                                                                                                                  |
| <b>IPv6-Adresse</b><br>-                                                                                                     | <b>Instance-Status</b><br>Läuft                                                        | <b>Öffentlicher DNS</b><br>ec2-100-24-242-107.compute-1.amazonaws.com   <a href="#">offene Adresse</a>                                                     |
| <b>Hostname-Typ</b><br>IP-Name: ip-10-0-1-50.ec2.internal                                                                    | <b>Private IP DNS-Name (nur IPv4)</b><br>ip-10-0-1-50.ec2.internal                     | <b>Elastic IP-Adressen</b><br>100.24.242.107 (tf-web-eip) [Öffentliche IP-Adresse]                                                                         |
| <b>DNS-Name der Antwort für private Ressourcen</b><br>-                                                                      | <b>Instance-Typ</b><br>t2.micro                                                        | <b>AWS Compute Optimizer-Treffer</b><br>Melden Sie sich für AWS Compute Optimizer an, um Empfehlungen zu erhalten.   <a href="#">Weitere Informationen</a> |
| <b>Automatisch zugewiesene IP-Adresse</b><br>-                                                                               | <b>VPC-ID</b><br>vpc-01642c1407e32f458 (tf-web-vpc)                                    | <b>Auto-Scaling-Gruppenname</b><br>-                                                                                                                       |
| <b>IAM-Rolle</b><br>-                                                                                                        | <b>Subnetz-ID</b><br>subnet-003febe3900aa2e0f (tf-web-subnet)                          | <b>Verwaltet</b><br>Falsch                                                                                                                                 |
| <b>IMDSv2</b><br>Optional<br>⚠ EC2 empfiehlt, IMDSv2 auf erforderlichlich zu setzen.   <a href="#">Weitere Informationen</a> | <b>Instance-ARN</b><br>arn:aws:ec2:us-east-1:767397790708:instance/i-0b44d8e0decc8c1f3 |                                                                                                                                                            |
| <b>Operator</b><br>-                                                                                                         |                                                                                        |                                                                                                                                                            |

Abbildung 10: EC2 Instance Summary

## 2.3 Execution

We apply the configuration and verify the creation of resources. The output confirms the public IP of our instance.

```

> terraform fmt
> terraform validate
Success! The configuration is valid.

> terraform apply -auto-approve
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 0s [id=ami-00b13f11600160c10]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_eip.web_eip will be created
+ resource "aws_eip" "web_eip" {
+   allocation_id      = (known after apply)
+   arn                = (known after apply)
+   associate_with_private_ip = "10.0.1.50"
+   association_id      = (known after apply)
+   carrier_ip         = (known after apply)
+   customer_owned_ip   = (known after apply)
+   domain             = "vpc"
+   id                 = (known after apply)
+   instance           = (known after apply)
+   ipam_pool_id        = (known after apply)
+   network_border_group = (known after apply)
+   network_interface   = (known after apply)
+   private_dns         = (known after apply)
+   private_ip          = (known after apply)
+   ptr_record          = (known after apply)
+   public_dns          = (known after apply)
+   public_ip           = (known after apply)
+   public_ipv4_pool     = (known after apply)
+   region              = "us-east-1"
+   tags                = {
+     "Name" = "tf-web-eip"
+   }
+   tags_all            = {
+     "Name" = "tf-web-eip"
+   }
}

```

Abbildung 11: Terraform Apply Execution

## 3 Workshop 3: Load Balancing and High Availability

In this workshop, we implement a highly available web application architecture. We distribute traffic across two EC2 instances in different Availability Zones (AZs) using an Application Load Balancer (ALB). This ensures that if one AZ fails, the application remains accessible.

### 3.1 Networking Setup

#### 3.1.1 VPC and Internet Gateway

We create a VPC and an Internet Gateway, similar to the previous workshop, to provide a network environment and internet connectivity.

```

resource "aws_vpc" "web_vpc" {
  cidr_block      = "10.0.0.0/16"

```

```

enable_dns_support    = true
enable_dns_hostnames = true
tags = { Name = "tf-web-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.web_vpc.id
  tags = { Name = "tf-web-igw" }
}

```

vpc-0d3e53f5fe370f9c8 / tf-web-vpc

Details | Ressourcenzuordnung | CIDRs | Flow-Protokolle | Tags | Integrationen

| Details                                            |                                                             |                                                                                              |                                                       |
|----------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------|
| <b>VPC-ID</b><br>vpc-0d3e53f5fe370f9c8             | <b>Status</b><br>Available                                  | <b>Öffentlichen Zugriff blockieren</b><br>Aus                                                | <b>DNS-Hostnamen</b><br>Aktiviert                     |
| <b>DNS-Auflösung</b><br>Aktiviert                  | <b>Tenancy</b><br>default                                   | <b>DHCP-Optionssatz</b><br>dopt-0804897584b01e432                                            | <b>Haupt-Routing-Tabelle</b><br>rtb-0c9d70a1e6b2db87c |
| <b>Haupt-Netzwerk-ACL</b><br>acl-014c8797788a6f9ab | <b>Standard-VPC</b><br>Nein                                 | <b>IPv4-CIDR</b><br>10.0.0.0/16                                                              | <b>IPv6-Pool</b><br>-                                 |
| <b>IPv6 CIDR (Network Border Group)</b><br>-       | <b>Nutzungsmetriken für Netzwerkadressen</b><br>Deaktiviert | <b>Regelgruppen für Route-53-Resolver-DNS-Firewall</b><br>Fehler beim Laden der Regelgruppen | <b>Besitzer-ID</b><br>767397790708                    |
| <b>Verschlüsselungssteuerungs-ID</b><br>-          | <b>Verschlüsselungssteuerungsmodus</b><br>-                 |                                                                                              |                                                       |

Abbildung 12: VPC Configuration

igw-0d6421b724ef21c6a / tf-web-igw

Details | Tags

| Details                                             |                           |                                                     |                                   |
|-----------------------------------------------------|---------------------------|-----------------------------------------------------|-----------------------------------|
| <b>Internet-Gateway-ID</b><br>igw-0d6421b724ef21c6a | <b>Status</b><br>Attached | <b>VPC-ID</b><br>vpc-0d3e53f5fe370f9c8   tf-web-vpc | <b>Eigentümer</b><br>767397790708 |

Abbildung 13: Internet Gateway

### 3.1.2 Subnets in Multiple AZs

To achieve high availability, we create two subnets in different Availability Zones (us-east-1a and us-east-1b).

```

resource "aws_subnet" "subnet_1" {
  vpc_id            = aws_vpc.web_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "us-east-1a"
  map_public_ip_on_launch = true
  tags = { Name = "tf-subnet-1" }
}

```

```

}

resource "aws_subnet" "subnet_2" {
  vpc_id            = aws_vpc.web_vpc.id
  cidr_block        = "10.0.2.0/24"
  availability_zone  = "us-east-1b"
  map_public_ip_on_launch = true
  tags = { Name = "tf-subnet-2" }
}

```

subnet-06da062a016089c28 / tf-subnet-1

| Details                                                         |                                                                                                          |                                                                  |                                                                             |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Subnetz-ID</b><br><a href="#">subnet-06da062a016089c28</a>   | <b>Subnetz-ARN</b><br><a href="#">arn:aws:ec2:us-east-1:767397790708:subnet/subnet-06da062a016089c28</a> | <b>Status</b><br><span>Available</span>                          | <b>Öffentlichen Zugriff blockieren</b><br><span>Aus</span>                  |
| <b>IPv4-CIDR</b><br><a href="#">10.0.1.0/24</a>                 | <b>Verfügbare IPv4-Adressen</b><br><a href="#">249</a>                                                   | <b>IPv6-CIDR</b><br>-                                            | <b>IPv6-CIDR-Zuordnungs-ID</b><br>-                                         |
| <b>Verfügbare Zone</b><br><a href="#">use1-az1 (us-east-1a)</a> | <b>Network Border Group</b><br><a href="#">us-east-1</a>                                                 | <b>VPC</b><br><a href="#">vpc-0d3e53f5fe370f9c8   tf-web-vpc</a> | <b>Routing-Tabelle</b><br><a href="#">rtb-071cf06657359c5ab   tf-web-rt</a> |
| <b>Netzwerk-ACL</b><br><a href="#">acl-014c8797788a6f9ab</a>    | <b>Standard-Subnetz</b><br>Nein                                                                          | <b>Öffentliche IPv4-Adresse automatisch zuweisen</b><br>Ja       | <b>IPv6-Adresse automatisch zuweisen</b><br>Nein                            |
| <b>Kundeneigene IPv4-Adresse automatisch zuweisen</b><br>Nein   | <b>Kundeneigener IPv4-Pool</b><br>-                                                                      | <b>Stützpunkt-ID</b><br>-                                        | <b>IPv4 CIDR-Reservierungen</b><br>-                                        |
| <b>IPv6 CIDR-Reservierungen</b><br>-                            | <b>Nur IPv6</b><br>Nein                                                                                  | <b>Hostname-Typ</b><br>IP-Name                                   | <b>Ressourcenname-DNS-A-Datensatz</b><br>Deaktiviert                        |
| <b>Ressourcenname-DNS-AAAA-Datensatz</b><br>Deaktiviert         | <b>DNS64</b><br>Deaktiviert                                                                              | <b>Eigentümer</b><br><a href="#">767397790708</a>                |                                                                             |

Abbildung 14: Subnet 1 in us-east-1a

subnet-0804b2f9df7fcbc67 / tf-subnet-2

| Details                                                         |                                                                                                          |                                                                  |                                                                             |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Subnetz-ID</b><br><a href="#">subnet-0804b2f9df7fcbc67</a>   | <b>Subnetz-ARN</b><br><a href="#">arn:aws:ec2:us-east-1:767397790708:subnet/subnet-0804b2f9df7fcbc67</a> | <b>Status</b><br><span>Available</span>                          | <b>Öffentlichen Zugriff blockieren</b><br><span>Aus</span>                  |
| <b>IPv4-CIDR</b><br><a href="#">10.0.2.0/24</a>                 | <b>Verfügbare IPv4-Adressen</b><br><a href="#">249</a>                                                   | <b>IPv6-CIDR</b><br>-                                            | <b>IPv6-CIDR-Zuordnungs-ID</b><br>-                                         |
| <b>Verfügbare Zone</b><br><a href="#">use1-az2 (us-east-1b)</a> | <b>Network Border Group</b><br><a href="#">us-east-1</a>                                                 | <b>VPC</b><br><a href="#">vpc-0d3e53f5fe370f9c8   tf-web-vpc</a> | <b>Routing-Tabelle</b><br><a href="#">rtb-071cf06657359c5ab   tf-web-rt</a> |
| <b>Netzwerk-ACL</b><br><a href="#">acl-014c8797788a6f9ab</a>    | <b>Standard-Subnetz</b><br>Nein                                                                          | <b>Öffentliche IPv4-Adresse automatisch zuweisen</b><br>Ja       | <b>IPv6-Adresse automatisch zuweisen</b><br>Nein                            |
| <b>Kundeneigene IPv4-Adresse automatisch zuweisen</b><br>Nein   | <b>Kundeneigener IPv4-Pool</b><br>-                                                                      | <b>Stützpunkt-ID</b><br>-                                        | <b>IPv4 CIDR-Reservierungen</b><br>-                                        |
| <b>IPv6 CIDR-Reservierungen</b><br>-                            | <b>Nur IPv6</b><br>Nein                                                                                  | <b>Hostname-Typ</b><br>IP-Name                                   | <b>Ressourcenname-DNS-A-Datensatz</b><br>Deaktiviert                        |
| <b>Ressourcenname-DNS-AAAA-Datensatz</b><br>Deaktiviert         | <b>DNS64</b><br>Deaktiviert                                                                              | <b>Eigentümer</b><br><a href="#">767397790708</a>                |                                                                             |

Abbildung 15: Subnet 2 in us-east-1b



### 3.1.3 Routing

We configure a route table to send internet traffic to the Internet Gateway and associate it with both subnets.

```
resource "aws_route_table" "web_rt" {
  vpc_id = aws_vpc.web_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "tf-web-rt" }
}

resource "aws_route_table_association" "rta_1" {
  subnet_id      = aws_subnet.subnet_1.id
  route_table_id = aws_route_table.web_rt.id
}

resource "aws_route_table_association" "rta_2" {
  subnet_id      = aws_subnet.subnet_2.id
  route_table_id = aws_route_table.web_rt.id
}
```

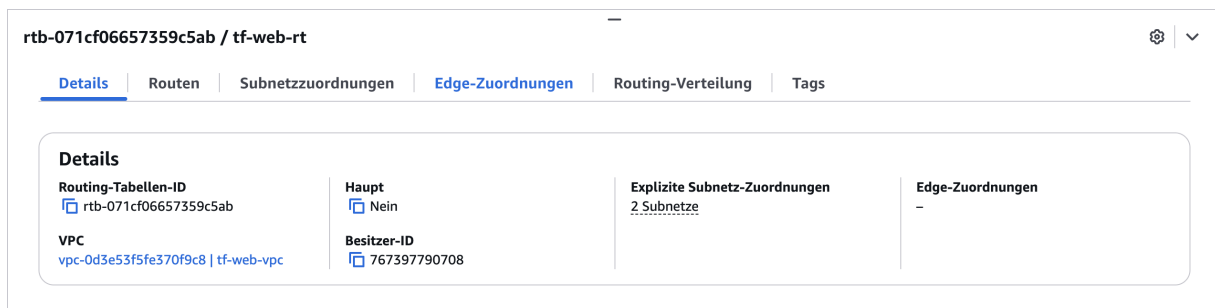


Abbildung 16: Route Table



Abbildung 17: Route Table Association

## 3.2 Security Groups

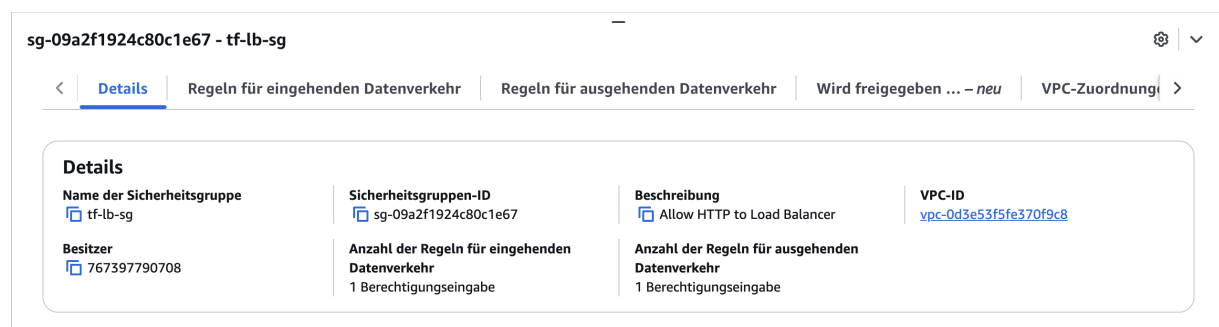
### 3.2.1 Load Balancer Security Group

This security group allows incoming HTTP traffic from anywhere to the Load Balancer.

```
resource "aws_security_group" "lb_sg" {
  name          = "tf-lb-sg"
  description   = "Allow_HTTP_to_Load_Balancer"
  vpc_id        = aws_vpc.web_vpc.id

  ingress {
    description = "HTTP_from_Internet"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```



The screenshot shows the AWS Management Console interface for the security group 'sg-09a2f1924c80c1e67 - tf-lb-sg'. The 'Details' tab is selected, showing the following information:

| Details                                       |                                                                                 |                                                                                 |                                        |
|-----------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------|
| <b>Name der Sicherheitsgruppe</b><br>tf-lb-sg | <b>Sicherheitsgruppen-ID</b><br>sg-09a2f1924c80c1e67                            | <b>Beschreibung</b><br>Allow HTTP to Load Balancer                              | <b>VPC-ID</b><br>vpc-0d3e53f5fe370f9c8 |
| <b>Besitzer</b><br>767397790708               | <b>Anzahl der Regeln für eingehenden Datenverkehr</b><br>1 Berechtigungseingabe | <b>Anzahl der Regeln für ausgehenden Datenverkehr</b><br>1 Berechtigungseingabe |                                        |

Abbildung 18: Load Balancer Security Group

### 3.2.2 EC2 Security Group

The EC2 instances should only accept traffic from the Load Balancer. We reference the Load Balancer's security group ID in the ingress rule.

```
resource "aws_security_group" "ec2_sg" {
  name = "tf-ec2-sg"
```

```

description = "Allow_HTTP_from_LB"
vpc_id      = aws_vpc.web_vpc.id

ingress {
  description      = "HTTP_from_Load_Balancer"
  from_port        = 80
  to_port          = 80
  protocol         = "tcp"
  security_groups  = [aws_security_group.lb_sg.id]
}

egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
}

```

**sg-03f1a7278cc31611e - tf-ec2-sg**

< **Details** | Regeln für eingehenden Datenverkehr | Regeln für ausgehenden Datenverkehr | Wird freigegeben ... - neu | VPC-Zuordnung >

| Details                                        |                                                                                 |                                                                                 |                                        |
|------------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------|
| <b>Name der Sicherheitsgruppe</b><br>tf-ec2-sg | <b>Sicherheitsgruppen-ID</b><br>sg-03f1a7278cc31611e                            | <b>Beschreibung</b><br>Allow HTTP from LB                                       | <b>VPC-ID</b><br>vpc-0d3e53f5fe370f9c8 |
| <b>Besitzer</b><br>767397790708                | <b>Anzahl der Regeln für eingehenden Datenverkehr</b><br>1 Berechtigungseingabe | <b>Anzahl der Regeln für ausgehenden Datenverkehr</b><br>1 Berechtigungseingabe |                                        |

Abbildung 19: EC2 Security Group restricted to LB

## 3.3 Load Balancer Configuration

### 3.3.1 Application Load Balancer

We create an external Application Load Balancer spanning our two subnets.

```

resource "aws_lb" "app_lb" {
  name                = "tf-web-lb"
  internal            = false
  load_balancer_type = "application"
  security_groups     = [aws_security_group.lb_sg.id]
  subnets            = [aws_subnet.subnet_1.id, aws_subnet.subnet_2.id]
  tags = { Name = "tf-web-lb" }
}

```

Load Balancer: tf-web-lb

Details

Listener und Regeln

Netzwerkzuordnung

Ressourcenkarte

Sicherheit

Überwachung

Integrationen

Attribute

Details

Load-Balancer-Typ

Anwendung

Status

Aktiv

VPC

[vpc-0d3e53f5fe370f9c8](#)

IP-Adresstyp des Load Balancers

IPv4

Schema

Internet-facing

Gehostete Zone

Z35SXDOTRQ7X7K

Availability Zones

[subnet-0804b2f9df7fcbc67](#) us-east-1b (use1-az2)
 [subnet-06da062a016089c28](#) us-east-1a (use1-az1)

Datum erstellt

2. Dezember 2025, 22:31 (UTC+01:00)

Load-Balancer-ARN

[arn:aws:elasticloadbalancing:us-east-1:767397790708:loadbalancer/app/tf-web-lb/40af0919d6641b14](#)

DNS-Name

[tf-web-lb-504647856.us-east-1.elb.amazonaws.com](#) (A-Datensatz)

Abbildung 20: Application Load Balancer

### 3.3.2 Target Group and Listener

The target group defines where to route requests. The listener listens on port 80 and forwards traffic to this target group.

```
resource "aws_lb_target_group" "web_tg" {
  name      = "tf-web-tg"
  port      = 80
  protocol  = "HTTP"
  vpc_id    = aws_vpc.web_vpc.id

  health_check {
    path            = "/"
    interval        = 30
    timeout         = 5
    healthy_threshold = 2
    unhealthy_threshold = 2
  }
}

resource "aws_lb_listener" "front_end" {
  load_balancer_arn = aws_lb.app_lb.arn
  port              = "80"
  protocol          = "HTTP"

  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.web_tg.arn
  }
}
```

17

**Zielgruppe: tf-web-tg**

Details | Ziele | Überwachung | Zustandsprüfungen | Attribute | Tags

**Details**

arn:aws:elasticloadbalancing:us-east-1:767397790708:targetgroup/tf-web-tg/c689c8a88b048a01

|                             |                                                   |                                  |                                                     |
|-----------------------------|---------------------------------------------------|----------------------------------|-----------------------------------------------------|
| <b>Zieltyp</b><br>Instance  | <b>Protokoll : Port</b><br>HTTP: 80               | <b>Protokollversion</b><br>HTTP1 | <b>VPC</b><br><a href="#">vpc-0d3e53f5fe370f9c8</a> |
| <b>IP-Adresstyp</b><br>IPv4 | <b>Load Balancer</b><br><a href="#">tf-web-lb</a> |                                  |                                                     |

|                      |                             |                 |                |                 |                |
|----------------------|-----------------------------|-----------------|----------------|-----------------|----------------|
| 2<br>Ziele insgesamt | 2<br>Fehlerfrei<br>0 Anomal | 0<br>Fehlerhaft | 0<br>Ungenutzt | 0<br>Anfänglich | 0<br>Auslassen |
|----------------------|-----------------------------|-----------------|----------------|-----------------|----------------|

► **Verteilung der Ziele nach Availability Zone (AZ)**  
Wählen Sie Werte in dieser Tabelle aus, um die entsprechenden Filter anzuzeigen, die auf die unten angegebene Tabelle registrierter Ziele angewendet werden.

Abbildung 21: Target Group

**Load Balancer: tf-web-lb**

< Details | **Listener und Regeln** | Netzwerkzuordnung | Ressourcenkarte | Sicherheit | Überwachung | Integrationen | Attribute >

**Listener und Regeln (1)** Info

Ein Listener prüft auf Verbindungsanfragen an seinem konfigurierten Protokoll und Port. Der vom Listener empfangene Datenverkehr wird gemäß der Standardaktion und etwaigen zusätzlichen Regeln weitergeleitet.

Listener filtern

| <input type="checkbox"/> | Protokoll: Port         | Standardaktion                                                                                                                                            | Regeln                  | ARN | Sicherheitsrichtlinie | Standard-SSL-/TLS |
|--------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|-----|-----------------------|-------------------|
| <input type="checkbox"/> | <a href="#">HTTP:80</a> | <ul style="list-style-type: none"> <li>Zur Zielgruppe weiterleiten<br/><a href="#">tf-web-tg</a>: 1 (100%)<br/>Klebrigkeit der Zielgruppe: Aus</li> </ul> | <a href="#">1 Regel</a> | ARN | Nicht zutreffend      | Nicht zutreffend  |

Abbildung 22: Listener Configuration

## 3.4 Compute Resources

### 3.4.1 EC2 Instances

We launch two EC2 instances. We use the `count` parameter to create multiple instances and the modulo operator to alternate them between the two subnets. The user data script is customized to display the instance number.

```
resource "aws_instance" "web" {
  count          = 2
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  subnet_id     = count.index % 2 == 0 ? aws_subnet.subnet_1.id : aws_subnet
    .subnet_2.id
  vpc_security_group_ids = [aws_security_group.ec2_sg.id]
```

```

user_data = <<-EOF
#!/bin/bash
sudo apt-get update
sudo apt-get install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
echo "<h1>Hello_from_Instance_${count.index}_1</h1>" | sudo tee /var/
    www/html/index.html
EOF

tags = { Name = "tf-web-instance-${count.index}_1" }
}

```

i-0342188f5d898d4ee (tf-web-instance-1)

Details

Status und Alarme

Überwachung

Sicherheit

Netzwerk

Speicher

Tags

▼ Instance-Zusammenfassung Info

Instance-ID

i-0342188f5d898d4ee

IPv6-Adresse

-

Hostname-Typ

IP-Name: ip-10-0-1-72.ec2.internal

DNS-Name der Antwort für private Ressourcen

-

Automatisch zugewiesene IP-Adresse

44.214.104.205 [Öffentliche IP-Adresse]

Öffentliche IPv4-Adresse

44.214.104.205 | offene Adresse

Instance-Status

Läuft

Private IP DNS-Name (nur IPv4)

ip-10-0-1-72.ec2.internal

Instance-Typ

t2.micro

VPC-ID

vpc-0d3e53f5fe370f9c8 (tf-web-vpc)

Private IPv4-Adressen

10.0.1.72

Öffentlicher DNS

ec2-44-214-104-205.compute-1.amazonaws.com | offene Adresse

Elastic IP-Adressen

-

AWS Compute Optimizer-Treffer

Melden Sie sich für AWS Compute Optimizer an, um Empfehlungen zu erhalten. | Weitere Informationen

Abbildung 23: EC2 Instance 1

i-01e10c1d7f5113f4c (tf-web-instance-2)

Details

Status und Alarme

Überwachung

Sicherheit

Netzwerk

Speicher

Tags

▼ Instance-Zusammenfassung Info

Instance-ID

i-01e10c1d7f5113f4c

IPv6-Adresse

-

Hostname-Typ

IP-Name: ip-10-0-2-153.ec2.internal

DNS-Name der Antwort für private Ressourcen

-

Automatisch zugewiesene IP-Adresse

98.93.191.240 [Öffentliche IP-Adresse]

Öffentliche IPv4-Adresse

98.93.191.240 | offene Adresse

Instance-Status

Läuft

Private IP DNS-Name (nur IPv4)

ip-10-0-2-153.ec2.internal

Instance-Typ

t2.micro

VPC-ID

vpc-0d3e53f5fe370f9c8 (tf-web-vpc)

Private IPv4-Adressen

10.0.2.153

Öffentlicher DNS

ec2-98-93-191-240.compute-1.amazonaws.com | offene Adresse

Elastic IP-Adressen

-

AWS Compute Optimizer-Treffer

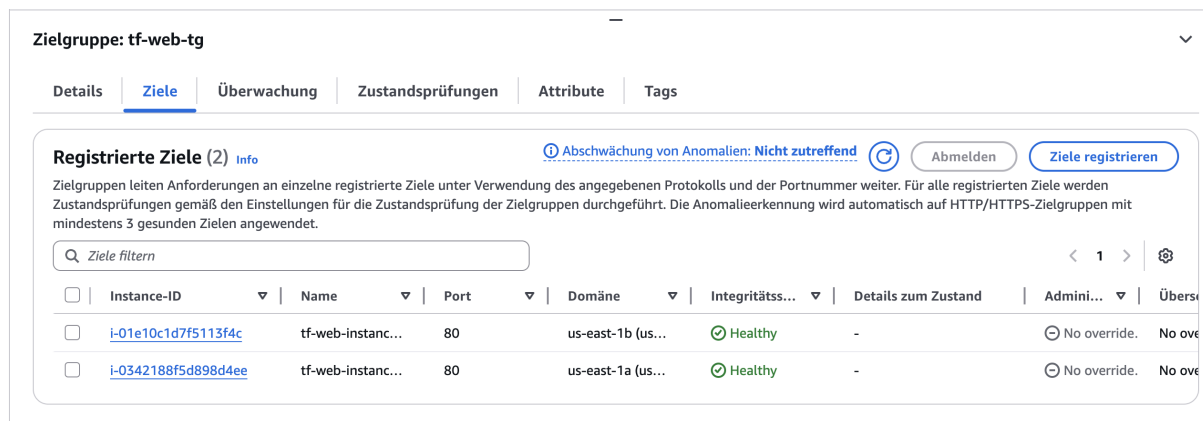
Melden Sie sich für AWS Compute Optimizer an, um Empfehlungen zu erhalten. | Weitere Informationen

Abbildung 24: EC2 Instance 2

### 3.4.2 Target Group Attachment

We attach the created instances to the target group so the load balancer can route traffic to them.

```
resource "aws_lb_target_group_attachment" "web_attach" {
  count          = 2
  target_group_arn = aws_lb_target_group.web_tg.arn
  target_id      = aws_instance.web[count.index].id
  port          = 80
}
```



**Zielgruppe: tf-web-tg**

Details | **Ziele** | Überwachung | Zustandsprüfungen | Attribute | Tags

**Registrierte Ziele (2)** [Info](#) [Abschwächung von Anomalien: Nicht zutreffend](#) [Abmelden](#) [Ziele registrieren](#)

Zielgruppen leiten Anforderungen an einzelne registrierte Ziele unter Verwendung des angegebenen Protokolls und der Portnummer weiter. Für alle registrierten Ziele werden Zustandsprüfungen gemäß den Einstellungen für die Zustandsprüfung der Zielgruppen durchgeführt. Die Anomalieerkennung wird automatisch auf HTTP/HTTPS-Zielgruppen mit mindestens 3 gesunden Zielen angewendet.

| <input type="checkbox"/> | Instance-ID                         | Name              | Port | Domäne            | Integrität... | Details zum Zustand | Admini...    | Übers... |
|--------------------------|-------------------------------------|-------------------|------|-------------------|---------------|---------------------|--------------|----------|
| <input type="checkbox"/> | <a href="#">i-01e10c1d7f5113f4c</a> | tf-web-instanc... | 80   | us-east-1b (us... | Healthy       | -                   | No override. | No ove   |
| <input type="checkbox"/> | <a href="#">i-0342188f5d898d4ee</a> | tf-web-instanc... | 80   | us-east-1a (us... | Healthy       | -                   | No override. | No ove   |

Abbildung 25: Target Group Attachment

## 3.5 Verification

After applying the configuration, we access the application via the Load Balancer's DNS name.

```
output "load_balancer_dns" {
  description = "The_DNS_name_of_the_load_balancer"
  value       = aws_lb.app_lb.dns_name
}
```

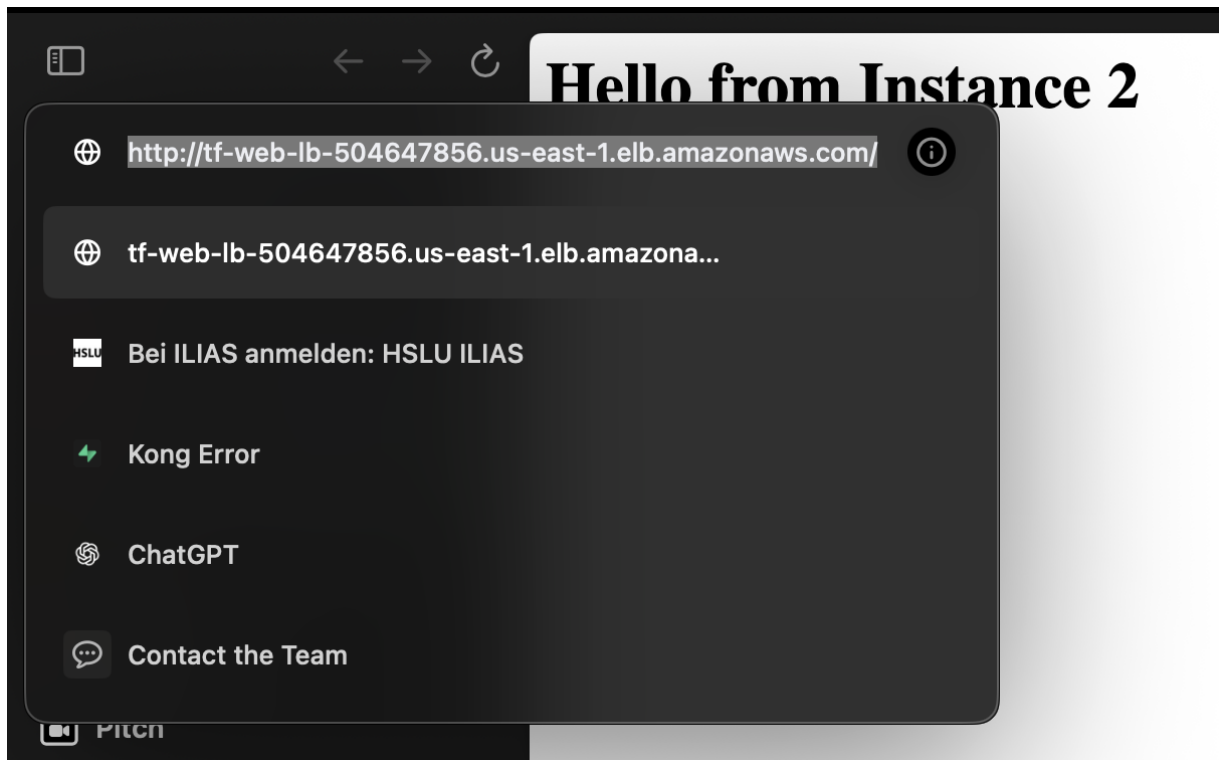


Abbildung 26: Website Accessed via Load Balancer

## 3.6 Automation and CI/CD

To automate the deployment process, we integrate GitHub Actions with Terraform Cloud. This allows us to automatically provision and manage our infrastructure upon code changes.

### 3.6.1 GitHub Configuration

We configure the necessary secrets in our GitHub repository to authenticate with Terraform Cloud.

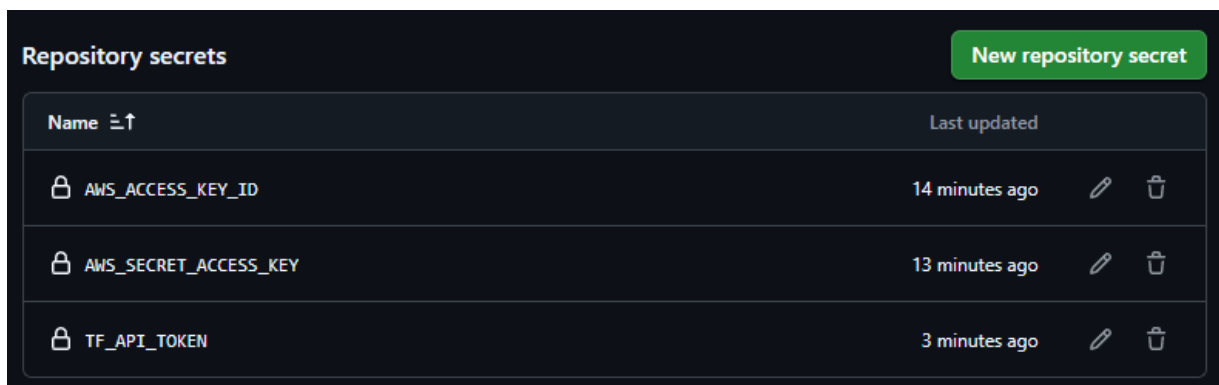


Abbildung 27: GitHub Repository Secrets



## 3.6.2 Terraform Cloud Configuration

In Terraform Cloud, we define the variables required for our infrastructure, such as AWS credentials and region settings.

### Workspace variables (3)

Variables defined within a workspace always overwrite variables from variable sets that have the same type and the same key. Learn more about variable set [precedence](#).

| Key                   | Value                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Category | Actions |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------|
| AWS_ACCESS_KEY_ID     | ASIA3FLDZAP2NU3F6LLY                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | env      | ...     |
| AWS_SECRET_ACCESS_KEY | C02gmFSowFD7fesiEHM3usYd8KmJ92Sqht<br>oeGosc                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | env      | ...     |
| AWS_SESSION_TOKEN     | IQoJb3JpZ2luX2VjEFeaCXVzLXdlc3QtMiJlM<br>EYCIQD32A57GWmbeCvDmH71p1eCnli25<br>e4E8L+8H41eZfo89glhAMWbPrKRO+dFlc4<br>JTOeTcWarw8RWAcNoIPA13b5IO74UKqYC<br>CBoQABoMNzY3Mzk3NzkwNzA4lgysp0V0<br>+91C2S6TAVYqgwlpB00+grqHu3U1IZLa6K<br>CPb0mEGbIYENUU40q/864udjJPFMkSnhf<br>W1IVFQ0Neh4h3u2JtVkyqdQNdmKDFmze<br>+PltxRg3S3r9cH6kUOw1YuKCOEzntBjFwdS<br>UUIgkBc/oSKp/HQC7QzHox7XgZCHm8Jtn<br>7CatNdNXdUesYZvE+XA6OUIAsZ3n5FoW6<br>pSqcWunal66SBU6HP1njK9UljFYeDC3HsF1<br>83waztkRJdyhqERZriwu1vDJeZIKgW4o3iR/<br>Nfo+5UNs86+ZT2wmd+0s3lkAfaQyxUb/z<br>gkC0Rmtu1kkY3hRwRc1GEILFDk11bke/Wf<br>ouZ9tmKBXNKkKbABU8M0K8Mle8vMkGO<br>pwBwGXtYkoNeST2RixlMNmiXpP7NXdH8j7<br>KU/dVA6GdEGbXkTSOqvGo5+WBittVq49M<br>MoZCHLmhT8Y6FPt+wTtp/pm3QILTU1kKE<br>XhdvXy2Rdlyb/X5ZcsRekPQtBN2mp4oEWS<br>RTCKwLyRCK4ztMfLYw4Ej/dvUZ+qrx9Lbn9<br>UybUjKtYfbbpMGWwhhIkqUW1ez2c2SRtheU<br>qtU0oXXGn | env      | ...     |

Abbildung 28: Terraform Cloud Variables

## 3.6.3 CI/CD Pipeline

When code is pushed to the repository, a GitHub Action is triggered. This action communicates with Terraform Cloud to plan and apply the changes.

Deploy Infrastructure

Deploy Infrastructure #6

Summary

Jobs

terraform-apply

Run details

Usage

Workflow file

terraform-apply

successful run in 4m-43s

Search logs

Set up job

Checkout Code

Setup Terraform

terraform init

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

The execution of the plan can be monitored in Terraform Cloud.


infrc\_workshop

infrc\_workshop / Workspaces / workshop-3 / Runs / run-chukDgwEa2veo1dT

Force unlock
New run

Workspaces
workshop-3
Overview
Runs
States
Search & Import
Variables
Settings

## workshop-3

ID: ws-n7oyNzK8qkApVhZ3

Add workspace description.

Running Resources 17 Tags 0 Terraform v1.14.0

Updated a few seconds ago

### Triggered via CLI

Plan & apply duration: 4 minutes

Resources changed: +16 ~0 -0 Actions: 0 invoked

swipswup triggered a run from CLI 16 minutes ago
Run Details

Plan finished
16 minutes ago
Resources: 16 to add, 0 to change, 0 to destroy

Started 16 minutes ago > Finished 16 minutes ago

+ 16 to create

Filter resources by address...
Filter by operation
Show data sources Terraform 1.14.0
Download raw log

|   |   |     |                                              |
|---|---|-----|----------------------------------------------|
| > | + | aws | aws_instance.web[0]                          |
| > | + | aws | aws_instance.web[1]                          |
| > | + | aws | aws_internet_gateway.igw                     |
| > | + | aws | aws_lb_listener.front_end                    |
| > | + | aws | aws_lb_target_group_attachment.web_attach[0] |
| > | + | aws | aws_lb_target_group_attachment.web_attach[1] |
| > | + | aws | aws_lb_target_group.web_tg                   |
| > | + | aws | aws_lb.app_lb                                |
| > | + | aws | aws_route_table_association.rta_1            |
| > | + | aws | aws_route_table_association.rta_2            |
| > | + | aws | aws_route_table.web_rt                       |
| > | + | aws | aws_security_group.ec2_sg                    |
| > | + | aws | aws_security_group.lb_sg                     |
| > | + | aws | aws_subnet.subnet_1                          |
| > | + | aws | aws_subnet.subnet_2                          |
| > | + | aws | aws_vpc.web_vpc                              |

Outputs
1 planned to change

Download Sentinel mocks
Sentinel mocks can be used for testing your Sentinel policies

Apply finished
12 minutes ago
Resources: 16 added, 0 changed, 0 destroyed

Comment
Leave feedback or record a decision.
Add comment

Abbildung 30: Terraform Cloud Apply Run

Finally, we can also manage the destruction of resources through the pipeline or Terraform Cloud directly.

Destroy Infrastructure #1

Summary

Jobs

terraform-deploy

Run details

Usage

Workflow file

terraform-deploy

Set up job

Checkout Code

Setup Terraform

Terraform Init

Terraform Destroy

terraform-deploy

successful row = 16/16

Search logs

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504



28

## 3.7 Source Code

All workshop files and the complete Terraform configuration can be found in the following GitHub repository: [https://github.com/SwipSwup/infrastructure\\_terraform](https://github.com/SwipSwup/infrastructure_terraform)